



VIT[®]
B H O P A L
www.vitbhopal.ac.in

Python Programming (CSE3011) |

LP (3 credits)

UNIT – 2

By

Dr. MK Jayanthi Kannan, B.E.,M.E.,MS.,MBA.,M.Phil., Ph.D.,
Professor School of Computing Science and Engineering,
AB 104, 8.30am- 5.pm

VIT Bhopal University, Bhopal-Indore Highway,
Kothrikalan, Madhya Pradesh - 466114.

email: jayanthi.m@vitbhopal.ac.in, **Emp Id : 100600**

Ph: +91 6379397594

Unit 2 Python Programming (CSE3011)

We are going to discuss the following topics using Python Programming Language in Unit 2,

- ✓ Conditions,
- ✓ Boolean logic,
- ✓ logical operators,
- ✓ ranges,
- ✓ Control statements: if-else, loops (for, while), Flow control,
- ✓ Functions, Scoping, Exceptions,
- ✓ Input and output,
- ✓ Modules, Collections,
- ✓ Lists, Tuples, Sets, Dictionaries,
- ✓ Modules, Standard Modules,
- ✓ Regular Expressions, Quantifiers,
- ✓ Basic String Operations

Course Code CSE3011	Python Programming	Course Type LP	Credits 3
Pre-requisite knowledge is required on Object-oriented programming paradigm and data structures.			
Course Objectives			
This course will introduce the Python Programming language, its functionality, code constructs, and its applications. This course is devised for following objectives, • To study object oriented paradigm in Python. • To develop their skill set using Python. • To familiarize with the functionalities and applications of Python.			
Course Outcomes			
Students will be able to solve problems, explore real-world software development challenges, and create practical and contemporary applications. At the completion of this course, students should be able to do the following: • Understand and use the Object Oriented paradigm in Python • Use the IO model in Python to read and write disk files. • Write Python programs using collections, regular expression, classifying and categorizing text.			

Unit No	Unit Content	No. of hours	SOs
1	A Brief History of Python, Different Versions, Python 2 vs Python 3, Installing Python, Environment Variables, Executing Python from the Command Line, Editing Python Files, Basic Python Syntax, String Values, String Operators, Numeric Data Types Conversions, Simple Input and Output, Language components - Control Flow structures and Syntax - Relational Operators - Logical Operators - Bit Wise Operators, Python for Windows	6	b,c,i
2	Conditions, boolean logic, logical operators, ranges, Control statements: if-else, loops (for, while), Flow control, Functions, Scoping, Exceptions, Input and output, Modules, Collections, Lists, Tuples, Sets, Dictionaries, Modules, Standard Modules, Regular Expressions, Quantifiers, Basic String Operations	6	c,i

Student Outcomes (SO): b, c, i, k, l

- b. An ability to analyze a problem, identify and define the computing requirements appropriate to its solution.
- c. An ability to design, implement and evaluate a system / computer-based system, process, component or program to meet desired needs
- i. Design and conduct experiment as well as analyze and interpret data.
- k. An ability to use current techniques, skills and tools necessary for computing engineering practice.
- l. An ability to apply mathematical foundations, algorithmic principles and computer science theory in the modeling and design of computer-based systems (CS)

3	Principles of Object Orientation, Classes in Python, Creating Classes, Instance Methods, Access Specification, data modeling, persistent storage of objects, inheritance, polymorphism, operator overloading, abstract classes, exception handling, try block	6	c,i
4	File Handling, Writing Data to a File, Reading Data From a File - Additional File Methods: Using Pipes as Data Streams, Handling	6	c,i
	IO Exceptions, Working with Directories, Metadata, File Organization, Database Programming - Generic Database Connectivity using ODBC, Postgres connection in Python, MySQL connection in Python.		

5	Graphical user interfaces, event-driven programming paradigm, tkinter module, creating simple GUI, buttons, labels, entry fields, dialogs, widget attributes - sizes, fonts, colors layouts, nested frames, Multithreading, Networks, and Client/Server Programming, introduction to HTML, interacting with remote HTML server, running html-based queries, downloading pages; CGI programming, programming a simple CGI form.	6	c,i,k
6	Guest Lecture on Contemporary Topics	02	
	Total Hours:	32	

Mode of Teaching and Learning: Flipped Class Room, Activity Based Teaching/Learning, Digital/Computer based models, wherever possible to augment lecture for practice/tutorial and minimum 2 hours lectures by industry experts on contemporary topics	
Mode of Evaluation and assessment: The assessment and evaluation components may consist of unannounced open book examinations, quizzes, student's portfolio generation and assessment, and any other innovative assessment practices followed by faculty, in addition to the Continuous Assessment Tests and Final Examinations.	
Text Books:	
1.	Mark Summerfield, "Programming in Python 3", 2nd Edition, Pearson Education, 2011.
2.	Harvey M. Deitel, "Python – How to program" , Prentice Hall, 2002
Reference Books:	
1.	Michael Dawson, "Python Programming for the Absolute Beginner", Third Edition, Cengage Learning, 2010.
2.	Mark Lutz, "Learning Python", 4th Edition, O'Reilly, 2000.

Text Books:	
1.	Mark Summerfield, “Programming in Python 3”, 2nd Edition, Pearson Education, 2011.
2.	Harvey M. Deitel, “Python – How to program” , Prentice Hall, 2002
Reference Books:	
1.	Michael Dawson, “Python Programming for the Absolute Beginner”, Third Edition, Cengage Learning, 2010.
2.	Mark Lutz, “Learning Python”, 4th Edition, O’Reilly, 2000.
Additional Recommended Reading	

List of Suggested Experiments:

1	Write python program to print list of numbers using range and for loop.
2	Write python program to print first n prime numbers.
3	Write python program to multiply matrices.
4	Write python program to take command line arguments (word count).
5	Write python program in which a function is defined and calling that function prints 'Hello World'.
6	Write python program to let user enter some data in string and then verify data and print welcome to user.
7	Write python program to store strings in list and then print them.
8	Write python program to find the most frequent words in a text read from a file.
9	Write python program in which a class is defined, then create object of that class and call simple 'print function' defined in class.
10	Write python program in which an function (with single string parameter) is defined and calling that function prints the string parameters given to function.
11	Simulate elliptical orbits in Pygame.
12	Simulate bouncing ball using Pygame.

What We Learn

- Input Output (Obtaining user Input)
- Control Flow: Conditional Statements if, if-else, nested elif
- Control Flow: Iterative Statement: while, for,
 - Iteration using range() function
 - Iteration over Sequence (list)
- Control Flow: Control Statements: break
- Strings/Lists: Slicing, len() function

What We Learn?

- ❑ Collections,
- ❑ Lists,
- ❑ Tuples,
- ❑ Sets,
- ❑ Dictionaries,
- ❑ Regular Expressions,
- ❑ Quantifiers
- ❑ Functions,
- ❑ Functions and Modules,
- ❑ Scoping,
- ❑ Exceptions
- ❑ Standard Modules

Control Flow: Conditional Statements: if else

□ #Program for testing input number and changing negative number to zero.

```
x = int(input("Please enter an integer: "))
```

```
if x < 0:
```

```
    x = 0
```

```
    print('Negative changed to zero')
```

```
elif x == 0:
```

```
    print('Zero')
```

```
elif x == 1:
```

```
    print('Single')
```

```
else:
```

```
    print('More')
```

Elif for two or more conditions

Control Flow: Iterative Statements:

while

**Write a Program that
uses while statement**

#output

```
#fib.py
```

1

```
#Fibonacci Series < 10
```

1

```
a, b = 0, 1
```

2

```
while b < 10:
```

3

```
    print(b)
```

5

```
    a, b = b, a+b
```

8

Control Flow: Iterative Statements: for

- The for statement in Python differs a bit from what you may be used to in C or Pascal. Rather than always iterating over an arithmetic progression of numbers (like in Pascal), or giving the user the ability to define both the iteration step and halting condition (as C).
- Python's for statement iterates over the items of any sequence (a list or a string), in the order that they appear in the sequence.

Control Flow: Iterative Statements: for

Program using for

```
#for statement  
words = ['cat', 'window',  
         'defene strate']  
for w in words:  
    print(w, len(w))
```

#output

```
cat 3  
window 6  
defene strate 13
```

Control Flow: Iterative Statements: for

Loop over a slice copy of the entire list.

```
words = ['cat', 'window',  
         'defenestrate']  
for w in words:  
    print(w, len(w))  
# Loop over a slice copy of  
# the entire list.  
for w in words[:]:  
    if len(w) > 6:  
        words.insert(0, w)  
print(words)
```

output

```
cat 3  
window 6  
defenestrate 12  
['defenestrate', 'cat',  
 'window', 'defenestrate']
```

Iteration using range() function

- If you do need to iterate over a sequence of numbers, the built-in function range() comes in handy.
- It generates arithmetic progressions:

#Program to use range() function

- for i in range(5):
 print(i)
- for j in range(5, 10):
 print(j)
- for k in range(0, 10, 3):
 print(k)
- for p in range(0, 10, 2):
 print(p)
- #check out the output

To iterate over the indices of a sequence,
you can combine range() and len() as follows

Program

```
a = ['Mary', 'had', 'a',  
     'little', 'lamb']  
for i in range(len(a)):  
    print(i, a[i])
```

#output

```
1 Mary  
2 had  
3 a  
4 little  
5 lamb
```



Control Flow: Control Statements: break

nestedloop.py

- #use of break-continue for n in range(2, 10):
 - for x in range(2, n): if n % x == 0:
 - print(n, 'equals', x, '*', n//x)
 - break
- # loop fell through without finding a factor
 - print(n, 'is a prime number')

3 is a prime number

4 equals 2 * 2

5 is a prime number 5 is a prime number

6 is a prime number

7 equals 2 * 3

8 is a prime number 7 is a prime number

7 is a prime number 7 is a prime number

9 is a prime number

10 equals 2 * 4

11 is a prime number 9 equals 3 * 3

Functions

- **Write a program that reminds you, to take breaks at certain intervals and connects you to your favourite YouTube channel, while working on your computer.**
 - **1. a code or function in the program that waits or calls at certain interval.**
 - **2. code to open the YouTube channel in browser.**
 - **3. Code to repeat the process (Loop).**

Program to demonstrate Functions

```
#Break Time program
```

```
import time
```

```
import webbrowser
```

```
breaks = 3
```

```
count = 1
```

```
print("This program started on "+time.ctime())
```

```
while(count<=breaks):
```

```
    time.sleep(05)
```

```
    webbrowser.open("https://www.youtube.com/watch  
    ?v=SAcPESN_Fk4")
```

```
count = count+1
```

Modules

- A module is a file containing Python definitions and statements.
- Python has a way to put definitions in a file and use them in a script or in an interactive instance of the interpreter. Such a file is called a *module*;
- The file name is the module name with the suffix `.py` appended.

Module

fib.py

```
def fib(n): # write
    Fibonacci series up to n
    a, b = 0, 1
    while b < n:
        print(b, end=' ')
        a, b = b, a+b
    print()
```

fib1.py

```
import fibo
fibo.fib(10)
```

#output

1

1

2

3

5

8

```
>>> fibonacci._name_  
'fibonacci'
```

□ If you intend to use a function often you can assign it to a local name:

```
>>> fib = fibonacci
```

```
>>> fib(500)
```

```
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Scoping

- LEGB Rule: LEGB (Local -> Enclosing -> Global -> Built-in) is the logic followed by a Python interpreter when it is executing your program.

```
# Global scope
x = 0

def outer():
    # Enclosed scope
    x = 1
    def inner():
        # Local scope
        x = 2
```



- Let's say you're calling `print(x)` within `inner()`, which is a function nested in `outer()`.
- Then Python will first look if "x" was defined locally within `inner()`. If not, the variable defined in `outer()` will be used. This is the enclosing function.
- If it also wasn't defined there, the Python interpreter will go up another level - to the global scope.
- Above that, you will only find the built-in scope, which contains special variables reserved for Python itself.

```
greeting = "Hello"
```

```
def change_greeting(new_greeting): greeting = new_greeting
```

```
def greeting_world():
```

```
    world = "World" print(greeting, world)
```

```
change_greeting("Hi") greeting_world()
```


Standard Modules

- **Python Standard Library:**
- Python comes with a library of standard modules, described in a separate document, the Python Library Reference (“Library Reference” hereafter).
- Some modules are built into the interpreter; these provide access to operations that are not part of the core of the language but are nevertheless built in, either for efficiency or to provide access to operating system primitives such as system calls.
- The set of such modules is a configuration option which also depends on the underlying platform.

Collections

- ❑ The collections module helps us populate and manipulate data structures efficiently.
- ❑ the collections module to help you work with tuples, dictionaries, and lists.
- ❑ We'll use namedtuples to create tuples with named fields, defaultdict to concisely group information in dictionaries, and deque to efficiently add elements to either side of a list-like object.

more on Lists

- The list data type has some more methods. Here are all of the methods of list objects:
- **list.append(x)**
 - Add an item to the end of the list. Equivalent to `a[len(a):] = [x]`.
- **list.extend(iterable)**
 - Extend the list by appending all the items from the iterable. Equivalent to `a[len(a):] = iterable`.
- **list.insert(i, x)**
 - Insert an item at a given position. The first argument is the index of the element before which to insert, so `a.insert(0, x)` inserts at the front of the list, and `a.insert(len(a), x)` is equivalent to `a.append(x)`.
- **list.remove(x)**
 - Remove the first item from the list whose value is equal to x. It raises a `ValueError` if there is no such item.
- **list.pop([i])**
 - Remove the item at the given position in the list, and return it. If no index is specified, `a.pop()` removes and returns the last item in the list. (The square brackets around the i in the method signature denote that the parameter is optional, not that you should type square brackets at that position. You will see this notation frequently in the Python Library Reference.)

More on Lists

- ❑ `list.clear()`
- ❑ Remove all items from the list. Equivalent to `del a[:]`.
- ❑ `list.index(x[, start[, end]])`
- ❑ Return zero-based index in the list of the first item whose value is equal to `x`. Raises a `ValueError` if there is no such item.
 - ❑ The optional arguments `start` and `end` are interpreted as in the slice notation and are used to limit the search to a particular subsequence of the list. The returned index is computed relative to the beginning of the full sequence rather than the `start` argument.
- ❑ `list.count(x)`
- ❑ Return the number of times `x` appears in the list.
- ❑ `list.sort(*, key=None, reverse=False)`
- ❑ Sort the items of the list in place (the arguments can be used for sort customization, see `sorted()` for their explanation).
- ❑ `list.reverse()`
- ❑ Reverse the elements of the list in place.
- ❑ `list.copy()`
- ❑ Return a shallow copy of the list. Equivalent to `a[:]`.

More on Lists

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
>>> fruits.count('apple')
2
>>> fruits.count('tangerine')
0
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4)  # Find next banana starting a position 4
6
>>> fruits.reverse()
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange']
>>> fruits.append('grape')
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange', 'grape']
>>> fruits.sort()
>>> fruits
['apple', 'apple', 'banana', 'banana', 'grape', 'kiwi', 'orange', 'pear']
>>> fruits.pop()
'pear'
```

More on Lists

- Using List as Stack: use `append()` and `pop()`

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
```


More on Lists

□ Using Lists as Queues

- It is also possible to use a list as a queue, where the first element added is the first element retrieved (“first-in, first-out”); however, lists are not efficient for this purpose. While appends and pops from the end of list are fast, doing inserts or pops from the beginning of a list is slow (because all of the other elements have to be shifted by one).

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")           # Terry arrives
>>> queue.append("Graham")         # Graham arrives
>>> queue.popleft()                # The first to arrive now leaves
'Eric'
>>> queue.popleft()                # The second to arrive now leaves
'John'
>>> queue                          # Remaining queue in order of arrival
deque(['Michael', 'Terry', 'Graham'])
```

Programming exercise

1. Given two lists [1, 2,3] and [3, 1, 4]. Write a program that prints the given output using given lists.

Output: [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]

such that: new list combines the elements of two lists if they are not equal:

2. Given a list of squares and cubes. Determine the multiple of 2 from both, to add it to the new list.

Tuples

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
>>> # Tuples may be nested:
... u = t, (1, 2, 3, 4, 5)
>>> u
((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))
>>> # Tuples are immutable:
... t[0] = 88888
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> # but they can contain mutable objects:
... v = ([1, 2, 3], [3, 2, 1])
>>> v
([1, 2, 3], [3, 2, 1])
```

Sets

- ❑ Every set element is unique (no duplicates) and must be immutable (cannot be changed). However, a set itself is mutable. We can add or remove items from it.
- ❑ We can perform set operations over sets.

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> print(basket)                                # show that duplicates have been removed
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket                            # fast membership testing
True
>>> 'crabgrass' in basket
False

>>> # Demonstrate set operations on unique letters from two words
...
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                            # unique letters in a
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                                        # letters in a but not in b
{'r', 'd', 'b'}
>>> a | b                                        # letters in a or b or both
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                                        # letters in both a and b
{'a', 'c'}
>>> a ^ b                                        # letters in a or b but not both
{'r', 'd', 'b', 'm', 'z', 'l'}
```

Dictionaries

- Dictionaries are sometimes found in other languages as “associative memories” or “associative arrays”.
- Unlike sequences, which are indexed by a range of numbers, dictionaries are indexed by *keys*, which can be any immutable type; strings and numbers can always be keys.
- Tuples (mutable) can be used as keys if they contain only strings, numbers, or tuples;
- You can't use lists as keys, since lists can be modified in place using index assignments, slice assignments, or methods like `append()` and `extend()`.

Dictionaries

□ To create empty dictionary: Use constructor

`dict = {}`

`d= dict([('song', 4139), ('music', 4127), ('art', 4098)])`

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']
4098
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False
```

More on Dictionaries

□ zip() function.

- To combine two lists of keys and values into dictionary
- To combine two tuples of keys and values into dictionary

□ To combine two dictionaries

```
keys = ['one', 'two', 'three']  
values = [1, 2, 3]  
d1 = dict(zip(keys, values))
```

```
d3 = {**d1,  
      **d2} print(d3)
```

□ update() function:

```
temp = dict() for k in keys:  
    temp.update({k:sample[k]})  
print(temp)
```

□ Programming Exercises:

- To print particular key-value pairs from a dictionary.
 - Get the required keys into list and then iterate....
 - Generate new dictionary, keep adding key-value pairs and print.

```
#to work with dictionaries
sample = {'one': 1, 'two': 2, 'three': 3, 'four': 4, 'five': 5}
print(sample)
for k, v in sample.items(): print(k, v)
# to print particular key-value pairs from dict. keys = ['one',
'five']
output = { k: sample[k] for k in keys} print(output)
#following will print two separate dictionaries. for k in keys:
d = {k:sample[k]} print(d)
```


Exceptions and Handling exceptions

- Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it.
- Errors detected during execution are called *exceptions* and are not unconditionally fatal: you will soon learn how to handle them in Python programs.
- Other possible exception is `NameError` when any undefined variable is directly used in formula e.g. `weight`.

Defining Clean-up Actions

#Exception Handling

```
try:
    age = int(input('Enter your age: '))
    weight = float(input('Enter your weight: '))
    index = weight/age
    print(index)
except ZeroDivisionError:
    print('Age cannot be Zero.')
except ValueError:
    print('Wrong value of age is entered.')
finally:
    print('End of program. Re-run the program')
    print('Everything is fine')
```

Regular Expressions

- A regular expression (or RE) specifies a set of strings that matches it;
 - the functions in this module let you check if a particular string matches a given regular expression or if a given regular expression matches a particular string, which comes down to the same thing.
- Python has a module named re to work with RegEx. Here's an example:
- Python has a built-in package called re, which can be used to work with Regular Expressions.

```
#Regular Expressions
```

```
import re
```

```
pattern = '^a...s$'
```

```
test_string = 'abyss'
```

```
result = re.match(pattern, test_string)
```

```
if result:print("Regular Expressions Search  
successful.")
```

```
else:print("Regular Expressions Search  
unsuccessful.")
```

```
===== RESTART:
```

```
C:/PYTHON/regexexp.py =====
```

```
Regular Expressions Search successful.
```


Application areas of Regular Expressions

- ❑ To develop validation frameworks/validation logic.
- ❑ To develop Pattern matching applications (ctrl-f in windows, grep in UNIX, etc)
- ❑ To develop Translators like compilers, interpreters, etc
- ❑ To develop digital circuits
- ❑ To develop communication protocols like TCP/IP, UDP, etc
- ❑ **Examples:**
- ❑ **Example 1:** We can write a regular expression to represent all mobile numbers
- ❑ **Example 2:** We can write a regular expression to represent all mail ids.

Regular Expressions

- The re module offers a set of functions that allows us to search a string for a match:

Function	Description
<u>findall</u>	Returns a list containing all matches
<u>search</u>	Returns a <u>Match object</u> if there is a match anywhere in the string
<u>split</u>	Returns a list where the string has been split at each match
<u>sub</u>	Replaces one or many matches with a string

- **compile():**
- re module contains a compile() function to compile a pattern into RegexObject. It returns a pattern object, if the pattern is valid, which can be used for searching.

Regular Expressions

❑ MetaCharacters

- ❑ Metacharacters are characters that are interpreted in a special way by a RegEx engine. Here's a list of metacharacters:

- ❑ `[] . ^ $ * + ? { } ( ) \ |`

❑ Specify Pattern Using RegEx

- ❑ To specify regular expressions, metacharacters are used. In the above example, `^` and `$` are metacharacters.

Regular Expressions

- **[] - Square brackets:** Square brackets specifies a set of characters you wish to match.
- **. – Period:** A period matches any single character (except newline '\n').
- The caret symbol ^ is used to check if a string starts with a certain character.
- **\$ - Dollar:** The dollar symbol \$ is used to check if a string ends with a certain character.
- *** - Star:** The star symbol * matches zero or more occurrences of the pattern left to it.
- **+ - Plus:** The plus symbol + matches one or more occurrences of the pattern left to it.
- **? - Question Mark:** The question mark symbol ? matches zero or one occurrence of the pattern left to it.
- **{ } – Braces:** Consider this code: {n, m}. This means at least n, and at most m repetitions of the pattern left to it.x
- **| - Alternation:** Vertical bar | is used for alternation (or operator).
- **() – Group:** Parentheses () is used to group sub-patterns. For example, (a|b|c)xz match any string that matches either a or b or c followed by xz
- **\ - Backslash:** Backslash \ is used to escape various characters including all metacharacters. For example, \\$a match if a string contains \$ followed by a. Here, \$ is not interpreted by a RegEx engine in a special way.

Regular Expressions

- ❑ **Special Sequences** make commonly used patterns easier to write. Here's a list of special sequences:
- ❑ `\A` - Matches if the specified characters are at the start of a string.
- ❑ `\b` - Matches if the specified characters are at the beginning or end of a word.
- ❑ `\B` - Opposite of `\b`. Matches if the specified characters are not at the beginning or end of a word.
- ❑ `\d` - Matches any decimal digit. Equivalent to `[0-9]`
- ❑ `\D` - Matches any non-decimal digit. Equivalent to `[^0-9]`
- ❑ `\s` - Matches where a string contains any whitespace character. Equivalent to `[ \t\n\r\f\v]`.
- ❑ `\w` - Matches any alphanumeric character (digits and alphabets). Equivalent to `[a-zA-Z0-9_]`. By the way, underscore `_` is also considered an alphanumeric character.
- ❑ `\W` - Matches any non-alphanumeric character. Equivalent to `[^a-zA-Z0-9_]`
- ❑ `\Z` - Matches if the specified characters are at the end of a string.

Example

```
import re
```

```
#findall method.
```

```
s = 'The rain in Spain. This is a Python program' pattern = 'program\Z'
```

```
result = re.findall(pattern, s) print(result)
```

```
x = re.findall('ai', s) print(x)
```

```
#split function
```

```
txt = "The rain in Spain" x = re.split("\s", txt) print(x)
```

```
import re
```

```
pin = input('Enter pin>') #pattern = [1-3][4-6][7-9][0]
```

```
x = re.match('\d{4}\Z', pin) if x:
```

```
    print('valid pin') else:
```

```
    print('Invalid pin')
```

```
import re
```

Example

```
#findall method.
```

```
s = 'The rain in Spain. This is a Python program' pattern = 'program\Z'
```

```
result = re.findall(pattern, s) print(result)
```

```
x = re.findall('ai', s) print(x)
```

```
#split function
```

```
txt = "The rain in Spain" x = re.split("\s", txt) print(x)
```

```
import re
```

```
pin = input('Enter pin>') #pattern = [1-3][4-6][7-9][0]
```

```
x = re.match('\d{4}\Z', pin) if x:
```

```
print('valid pin') else:
```

```
print('Invalid pin')
```

□ We can use character classes to search for a group of characters

[abc]	:	Either a or b or c
[^abc]	:	Except a and b and c
[a-z]	:	Any Lower-case alphabet symbol
[A-Z]	:	Any Upper-case alphabet symbol
[a-zA-Z]	:	Any alphabet symbol
[0-9]	:	Any digit from 0 to 9
[a-zA-Z0-9]	:	Any alphanumeric character
[^a-zA-Z0-9]	:	Except alphanumeric characters(Special Characters)

Quantifiers

□ Quantifiers in Python:

- A quantifier has the form $\{m, n\}$ where m and n are the minimum and maximum times the expression to which the quantifier applies must match. We can use quantifiers to specify the number of occurrences to match.

a	:	Exactly one 'a'
a+	:	At least one 'a'
a*	:	Any number of a's including zero number
a?	:	At most one 'a' i.e either zero number or one number
a{m}	:	Exactly m number of a's
a{m,n}	:	Minimum m number of a's and Maximum n number of a's

Unit 2 Summary Python Programming (CSE3011)

We discussed the following topics and the Python programs using the concepts in

- ✓ CONDITIONS,
- ✓ BOOLEAN LOGIC,
- ✓ LOGICAL OPERATORS,
- ✓ RANGES,
- ✓ CONTROL STATEMENTS: IF-ELSE, LOOPS (FOR, WHILE), FLOW CONTROL,
- ✓ FUNCTIONS, SCOPING, EXCEPTIONS,
- ✓ INPUT AND OUTPUT,
- ✓ MODULES, COLLECTIONS,
- ✓ LISTS, TUPLES, SETS, DICTIONARIES,
- ✓ MODULES, STANDARD MODULES,
- ✓ REGULAR EXPRESSIONS,
- ✓ QUANTIFIERS,
- ✓ BASIC STRING OPERATIONS

Thank You..